S

SOFTWARE ENGINEERING LAB · PROJECT REPORT

StockIt

A web-based Inventory Management System for small businesses

Submitted by	
Roll number	
Course	Software Engineering Lab
Date	November 2026
Repository	github.com/Somasekhar711/IMS

React · Express · Node.js · PostgreSQL · Jest · Selenium · JUnit 5

Contents

1	Project Title, Aim and Objectives	3
2	SRS Document	3
3	Development Methodology	4
4	Sprint Details and Scrum Meetings	4
5	Technology Stack and Tools Used	5
6	System Design and UML Diagrams	6
7	Implementation and Modules	12
8	Software Testing and Test Cases	12

FIGURES

6.1	Use case diagram	6
6.2	Class diagram	8
6.3	Sequence diagram for creating a sales order	9
6.4	Activity diagram for sign in	10
6.5	Entity-relationship diagram of the database	11

1. Project Title, Aim and Objectives

Title: StockIt — a web-based Inventory Management System.

Aim: To give a small business one reliable place to record its products, stock, purchases and sales, so that stock levels are always correct and every change can be traced.

Objectives:

1. Maintain a product catalogue with price, tax (GST), category, expiry date and stock threshold.
2. Update stock automatically when goods are purchased or sold, and refuse a sale that exceeds the stock available.
3. Warn the user about low-stock, out-of-stock and soon-to-expire products.
4. Keep a ledger of every stock movement and a log of who performed each action.
5. Let the owner create employee logins with separate permissions for each module.
6. Keep each business's data completely separate from every other account.
7. Verify the system with automated unit, integration and browser tests.

2. SRS Document

Purpose and scope. StockIt is used through a web browser by a shop or warehouse owner (admin) and the owner's employees. It covers catalogue, inventory, suppliers, customers, purchase orders, sales orders, reports and user management. Billing, payments and accounting are outside its scope.

Users. *Admin:* registers the business account and has full access. *Employee:* signs in with a warehouse ID and employee ID, and sees only the modules the admin allows.

Functional requirements

ID	Requirement
FR1	Register, sign in and sign out; reset a forgotten password by answering a security question
FR2	Add, view, search, update and delete products; a product code (HSN) is unique within an account
FR3	Create, rename and delete categories; a category in use cannot be deleted
FR4	Adjust stock manually with a reason; show healthy, low and out-of-stock status
FR5	Maintain supplier and customer directories
FR6	Record a purchase order; its quantities are added to stock
FR7	Record a sales order; its quantities are removed from stock, and the order is refused if stock is insufficient
FR8	Deleting an order reverses its effect on stock
FR9	Show a ledger of all stock movements, filterable by product and type

ID	Requirement
FR10	Show reports: sales and purchase totals, top-selling products, stock health, for 7, 30 or 90 days
FR11	Admin creates employee logins and sets none, read or write permission for each module
FR12	Record every create, update and delete in an activity log visible to the admin

Non-functional requirements

Area	Requirement
Security	Passwords and security answers stored as bcrypt hashes; every request carries a signed token (JWT); permissions checked on the server; passwords need 8+ characters with a capital letter, a number and a special character
Data integrity	Orders and their stock changes are saved in one database transaction; database constraints forbid negative stock or price
Privacy	A user can never read or change another account's data
Usability	Responsive layout for desktop and mobile; clear error messages
Reliability	Invalid input returns a clear message, never a server crash
Maintainability	One file per module on both client and server; numbered database migrations; automated tests

Constraints. A modern browser, Node.js 18 or later and PostgreSQL 14 or later.

3. Development Methodology

The project followed **Agile Scrum**. Scrum suited the project because the requirements grew as it went: it began as a login page and a product list, and suppliers, orders, reports and employee access were added in later iterations. Working software was available at the end of every sprint.

- **Roles:** a product owner who ordered the backlog, a scrum master who ran the meetings and removed blockers, and the development team.
- **Artefacts:** a product backlog of user stories, a sprint backlog for each sprint, and a working increment merged into the `main` branch.
- **Practices:** feature branches and pull requests on GitHub, code review before merging, descriptive commit messages (`feat:`, `fix:`, `docs:`, `test:`), and numbered database migrations so every member's database stays in step.

4. Sprint Details and Scrum Meetings

The sprint contents below follow the project's commit history.

Sprint	Period (2026)	Goal	Delivered
1	13 Aug – 27 Aug	Foundation and interface	Repository and project structure; sign-in and registration screens with password rules; dashboard layout; first product page
2	28 Aug – 6 Sep	Working core	Express API and database schema; authentication with JWT; product create, read, update, delete; inventory page with stock adjustment; data isolated per account; review fixes
3	26 Sep – 8 Oct	Business modules	Categories, suppliers, customers; purchase and sales orders that update stock; stock movement ledger; reports; settings; password recovery by security question
4	29 Oct – 5 Nov	Multi-user access	Employee logins with a warehouse ID; per-module permissions; activity log; code audit fixes; date-handling fix
5	6 Nov – 11 Nov	Quality	181 automated tests with Jest, Selenium and JUnit; five defects found and fixed; test documentation; demo data

Scrum meetings

Meeting	When	What happened
Sprint planning	Start of each sprint	Chose user stories from the backlog, split them into tasks and assigned them
Daily stand-up	Daily, about 10 minutes	Each member answered: what did I finish, what will I do next, what is blocking me
Sprint review	End of each sprint	Demonstrated the working features and collected feedback
Sprint retrospective	After the review	Agreed what went well and what to change

Examples of retrospective outcomes: after Sprint 2, merge conflicts led the team to use feature branches and pull requests; after Sprint 3, email-based OTP sign-in was replaced by a security question because it depended on an outside email service; after Sprint 4, the absence of automated tests led to a dedicated testing sprint.

5. Technology Stack and Tools Used

Layer	Technology	Purpose
Front end	React, Vite, lucide-react icons, CSS	Single-page user interface
Back end	Node.js, Express	REST API
Database	PostgreSQL	Data storage, constraints and transactions
Security	bcrypt, JSON Web Tokens	Password hashing and authentication

Layer	Technology	Purpose
Testing	Jest, Supertest, Selenium WebDriver, JUnit 5	Unit, integration, browser and black-box tests
Tools	Git and GitHub, VS Code, npm, Microsoft Edge	Version control, editing, packages, test browser

6. System Design and UML Diagrams

Architecture. StockIt has three tiers. The React client runs in the browser and calls the API. The Express API authenticates the token, checks the role and permission, validates the input and runs SQL. PostgreSQL stores the data. The browser never talks to the database directly.

Use case diagram. Actors: *Admin* and *Employee*.

- Admin: register, sign in, reset password, manage products, manage categories, adjust stock, manage suppliers and customers, create purchase orders, create sales orders, view stock movements, view reports, manage employees and permissions, view activity log, change settings.
- Employee: sign in, change password, and any of the product, inventory, partner, order and report use cases that the admin has permitted.
- "Create sales order" *includes* "check stock"; "manage employees" *includes* "set permissions".

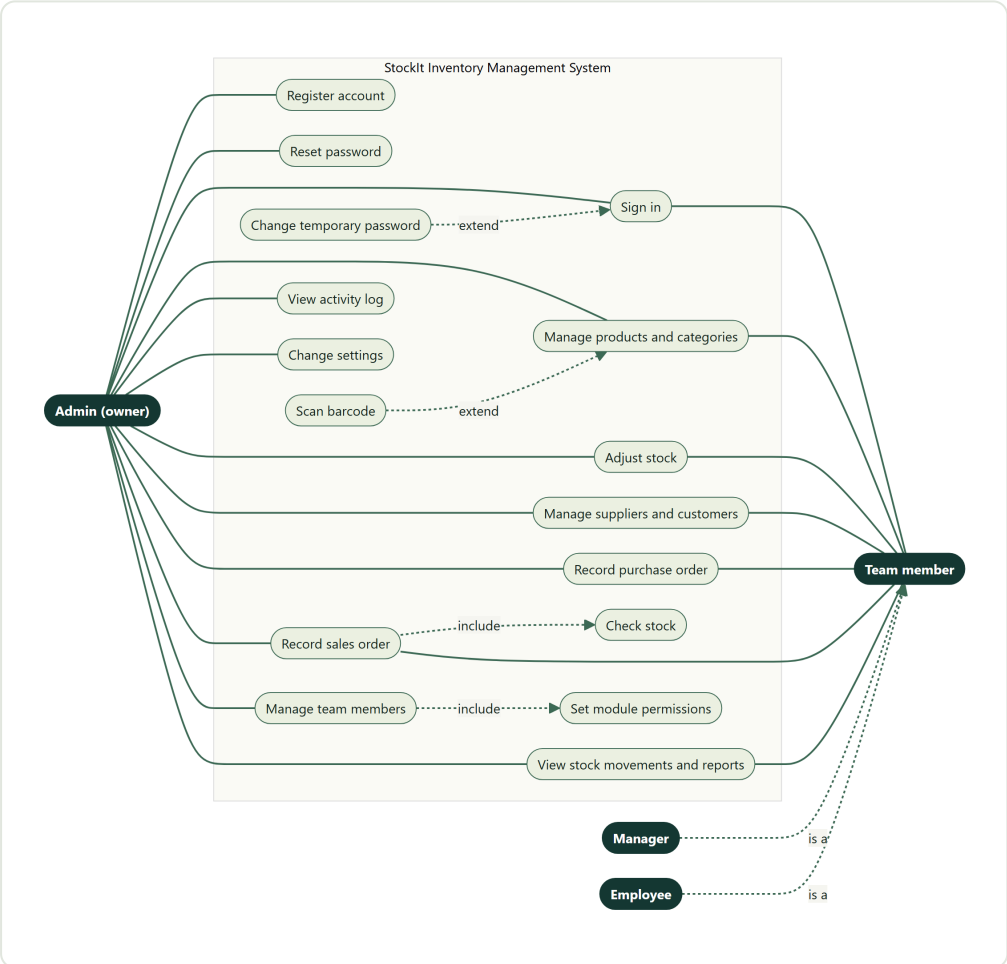


Figure 6.1. Use case diagram

Class diagram (main entities)

Class	Key attributes	Relationships
User	id, fullName, email, passwordHash, warehouseId	owns all other entities (1 to many)
TeamMember	id, name, empId, role, status, permissions	belongs to one User
Category	id, name, description	has many Products
Product	id, hsn, barcode, itemName, price, costPrice, gst, stockPresent, thresholdStock, expiryDate	belongs to one Category; supplied by many Suppliers
Supplier / Customer	id, name, phone, email, address	Supplier has many PurchaseOrders; Customer has many SalesOrders
PurchaseOrder / SalesOrder	id, orderDate, reference, totalAmount	each has many OrderItems (composition)
PurchaseOrderItem / SalesOrderItem	productId, quantity, unit cost or price, lineTotal (sales lines also keep the unit cost at the time of sale)	refers to one Product
InventoryMovement	id, type, quantityChange, stockAfter, createdAt	refers to one Product
ActivityLog	id, actorName, action, module, description	belongs to one User

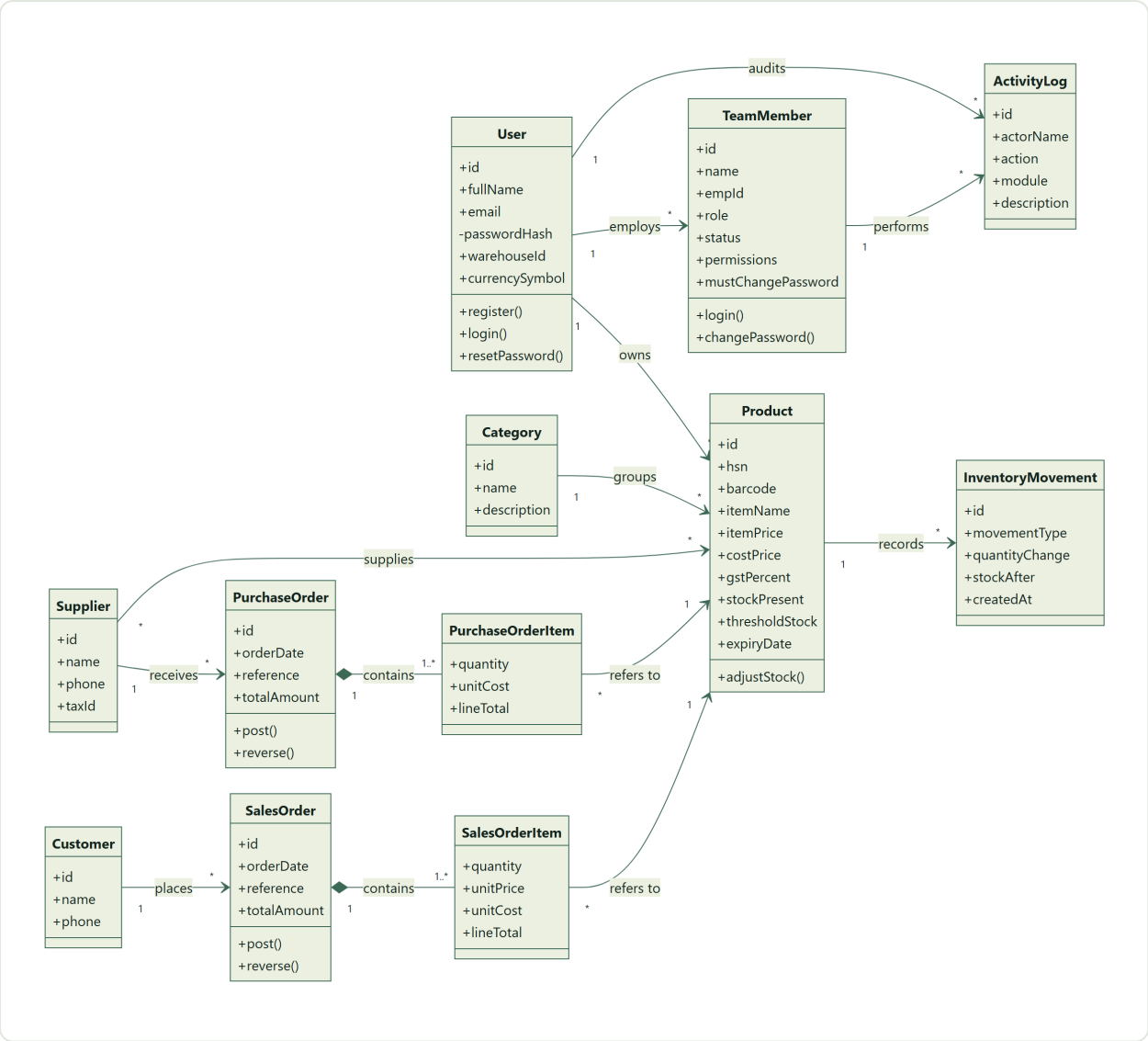


Figure 6.2. Class diagram

Sequence diagram — create sales order. (1) The user submits the order form. (2) The client sends `POST /api/sales-orders` with the token. (3) The API verifies the token and the "sales orders: write" permission. (4) The API begins a transaction and reads the stock of each product. (5) If any quantity exceeds stock, it rolls back and returns an error. (6) Otherwise it inserts the order and its items, reduces stock, writes ledger entries and commits. (7) The API writes the activity log and returns the order. (8) The client shows a success message and the new stock.

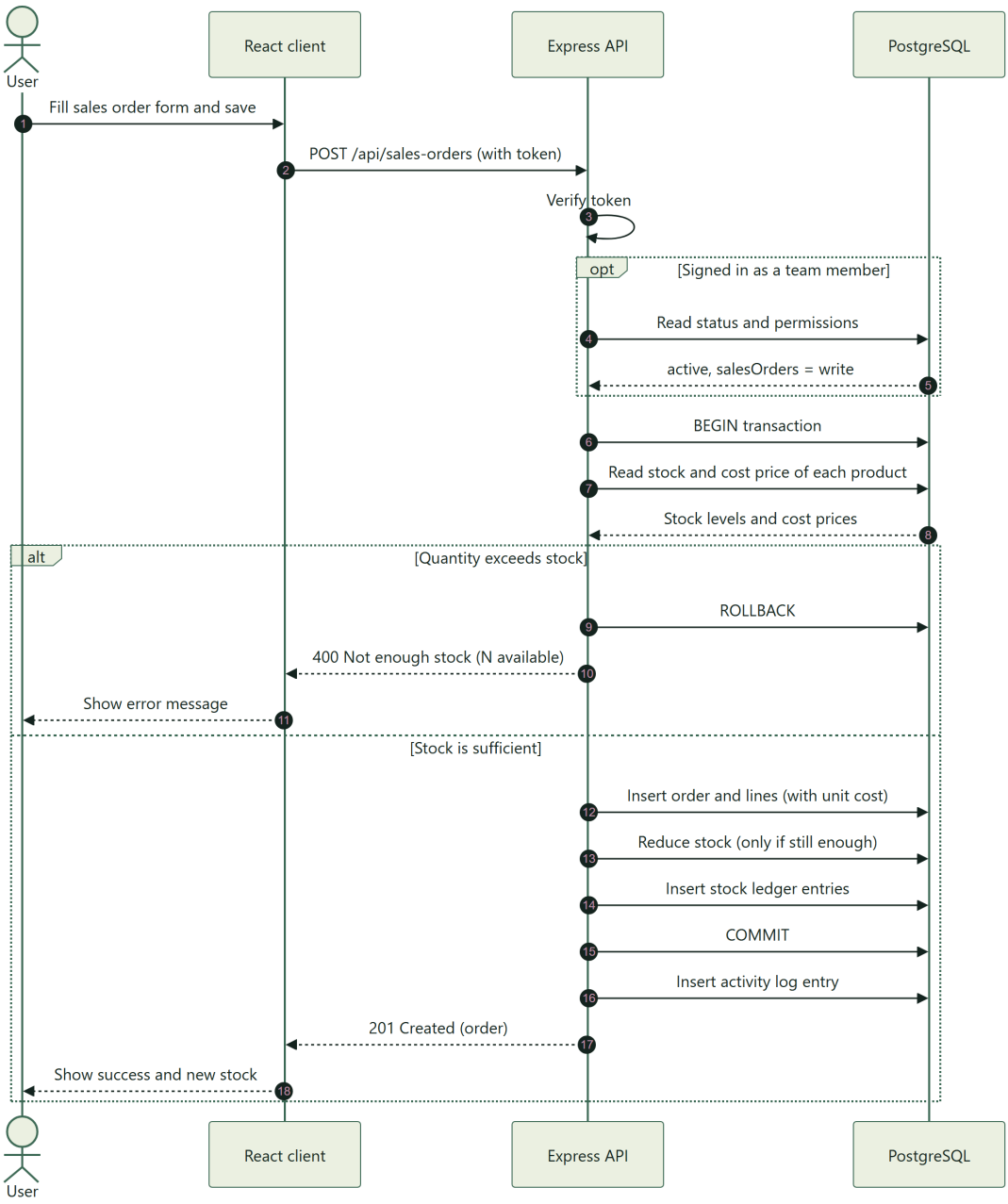


Figure 6.3. Sequence diagram for creating a sales order

Activity diagram — sign in. Enter credentials → validate fields → look up account → compare password hash → if wrong, show error and return to the form; if correct, issue token → open dashboard.

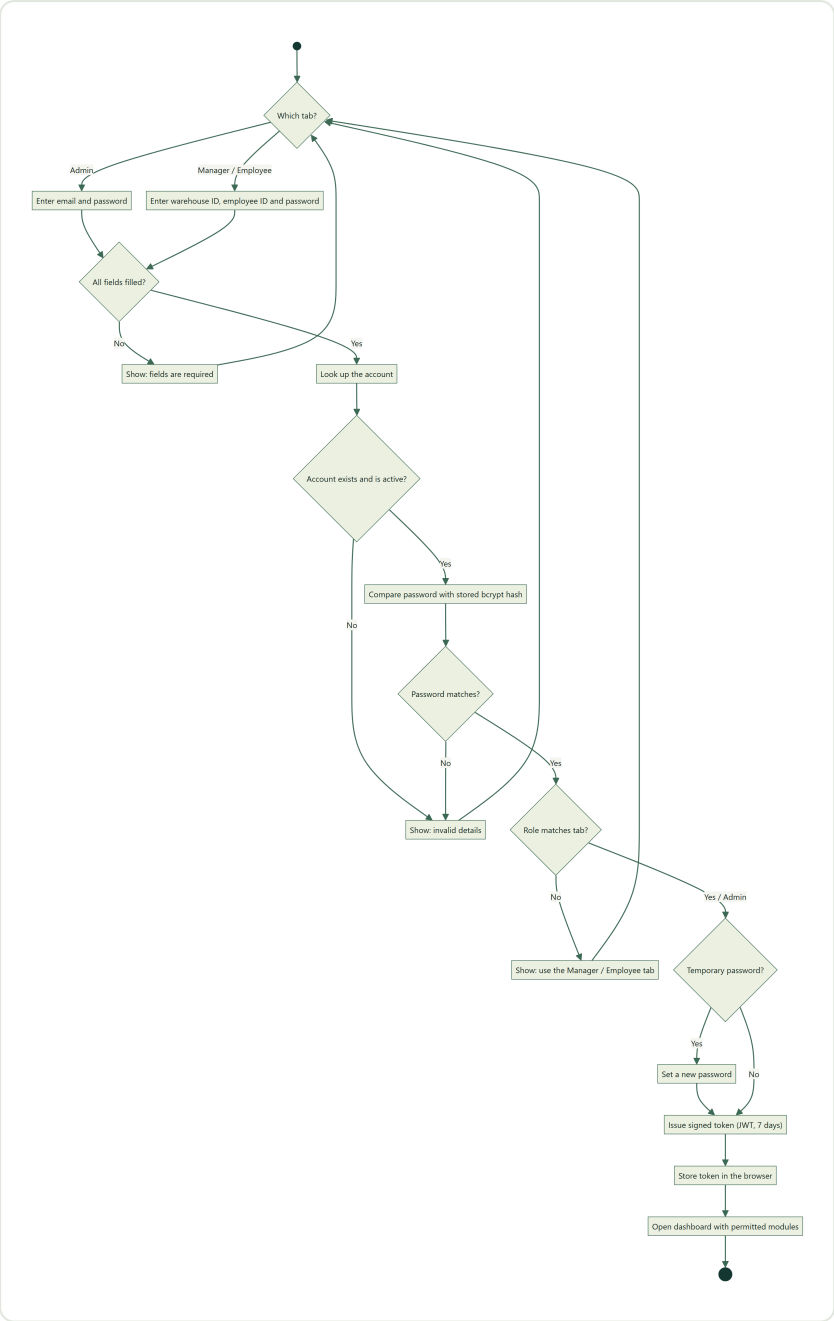


Figure 6.4. Activity diagram for sign in

Database design. Thirteen tables in third normal form: `users`, `team_members`, `categories`, `products`, `suppliers`, `supplier_products`, `customers`, `purchase_orders`, `purchase_order_items`, `sales_orders`, `sales_order_items`, `inventory_movements` and `activity_logs`. Every table carries `owner_user_id`, which enforces data isolation.

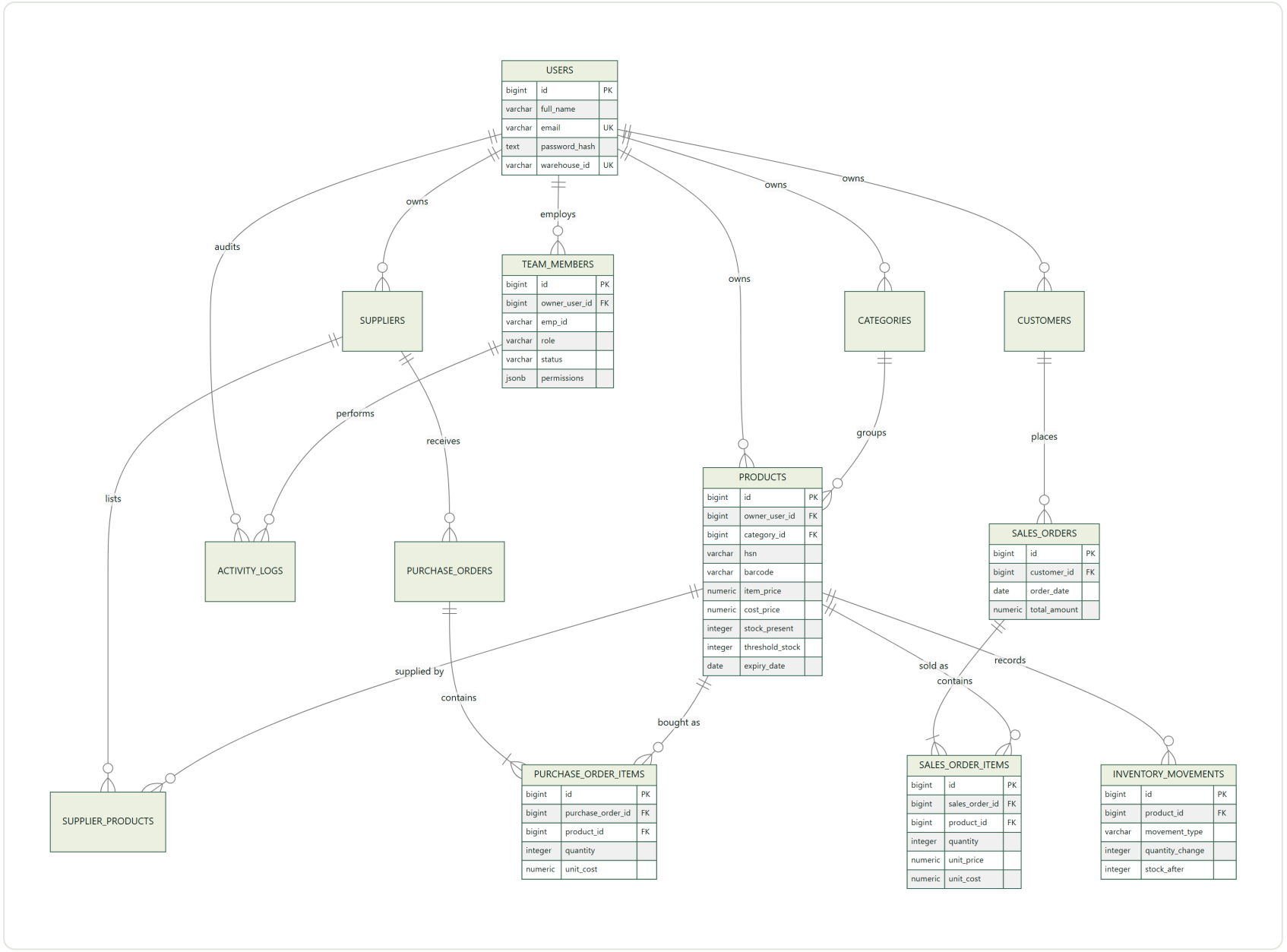


Figure 6.5. Entity-relationship diagram of the database

7. Implementation and Modules

Module	What it does
Authentication	Registration, sign-in, security-question password reset, JWT sessions
Dashboard	Summary cards (products, stock units, value, low stock, out of stock, expiring soon), activity feed, alerts, search
Products and Categories	Catalogue management; typing a new category on a product creates it
Inventory	Stock status and manual adjustments with a reason
Suppliers and Customers	Contact directories; suppliers are linked to the products they supply
Purchase Orders	Record goods received; adds to stock
Sales Orders	Record goods sold; removes from stock; blocks overselling
Stock Movements	Read-only ledger of every purchase, sale and adjustment
Reports	Totals, top sellers and stock health for a chosen period
Users and Roles	Employee logins, per-module permissions, activity log
Settings	Profile, password, security question, currency symbol, alert preference

Implementation highlights. All SQL uses parameterised queries, which prevents SQL injection. Stock-changing operations run inside database transactions, so an order is saved completely or not at all. Employee permissions are re-read from the database on every request, so a change by the admin takes effect immediately. The code is organised as one route file per module on the server and one page file per module on the client.

8. Software Testing and Test Cases

Strategy. Testing was automated at four levels. Every run starts a fresh, empty database, so results are repeatable.

Level	Tool	Tests	What is checked
Unit	Jest	27	Single functions: password policy, permission rules, token checks
Integration	Jest + Supertest	120	Every API route against a real PostgreSQL database
System	Selenium WebDriver	25	Complete user journeys in the Microsoft Edge browser

Level	Tool	Tests	What is checked
Black-box API	JUnit 5	9	The running system over HTTP from a Java client

Techniques used: equivalence partitioning, boundary value analysis, negative testing, security testing, state-based testing and regression testing.

Sample test cases

ID	Scenario	Input	Expected result	Status
IT-15	Sign in with wrong password	Valid email, wrong password	Error "Invalid email or password" (401)	Pass
UT-23	Password length boundary	7 characters, then 8	7 rejected, 8 accepted	Pass
IT-31	Invalid product data	Price −5	Rejected with a clear message (400)	Pass
IT-59	Overselling	Sell 7 when 6 are in stock	Refused; stock stays 6	Pass
IT-43	Access to another account	User B deletes user A's product	Not found (404); product still exists	Pass
IT-76	Read-only employee	Employee tries to add a product	Forbidden (403)	Pass
ST-17	Sale through the browser	Sell 5 of 55 using the form	Success message; stock shows 50	Pass
JU-08	Stock rule from an outside client	Sale of 4, then of 7, from stock of 10	First accepted, second refused; stock is 6	Pass

Results. All 181 tests pass. The tests execute 70% of the server's lines and 86% of its functions. Testing found five defects, all corrected and re-tested: invalid product data and non-numeric IDs caused server errors; a malformed request exposed internal error details; password rules were enforced only in the browser; and email addresses were treated as case-sensitive.

The complete list of 163 test cases is in `docs/TESTING.md`.